

Chapter 6 — Mapper and Context (helpers/)

Files:

- `src/features/react-crud-v4/helpers/mapItemDtoToRow.ts`
- `src/features/react-crud-v4/helpers/buildEditContext.ts`

One job: Pure functions that translate data between layers. No React, no side effects, trivially testable.

6.1 What is a Pure Function?

A **pure function** is a function that:

1. Given the same input, always returns the same output
2. Has no side effects (doesn't modify external state, doesn't call APIs, doesn't write to the DOM)

```
// Pure function – same input always gives same output, no side effects
const double = (n: number) => n * 2;
double(5); // always 21
double(5); // always 21

// Impure function – depends on external state
let multiplier = 2;
const multiply = (n: number) => n * multiplier;
multiply(5); // 10 today
multiplier = 3;
multiply(5); // 15 tomorrow – different output, same input!
```

Why do we care? Pure functions are:

- **Trivially testable** — no mocking, no setup, no teardown. Just call the function with input and check the output.
- **Predictable** — you can reason about them in isolation without understanding the rest of the system
- **Composable** — you can chain them together safely

Both `mapItemDtoToRow` and `buildEditContext` are pure functions. This is intentional.

6.2 `mapItemDtoToRow.ts` — Line by Line

```
import dayjs from "dayjs";
import type { ItemDto } from "../Services/crud.dtos";
```

dayjs — a lightweight date formatting library. The API sends ISO date strings like `"2024-03-15T00:00:00"`. **dayjs** converts them to human-readable formats like `"15/03/2024"`. Much simpler than the built-in `Date` API

for formatting.

`import type { ItemDto }` — we only need the TypeScript type here, not any runtime value. `import type` is a performance optimization.

The `ItemRow` Type

```
export type ItemRow = {
  id: number;           // StudentNumber
  studentKey: string;
  name: string;         // StudentName
  year: string;         // CurrentYear
  nsn: string;
  homeClass: string;
  coreClass: string;
  house: string;
  homeRoom: string;
  homeTeacher: string;
  dean: string;
  deputy: string;
  previousTermStatusId: number;
  currentTermStatusId: number;
  currentTermStatus: string;
  currentTermStatusColor: string;
  previousTermStatus: string;
  previousTermStatusColor: string;
  currentSummary: string;
  currentSummaryDetail: string;
  thresholdCrossed: string | null;
  daysAbsentT1: number;
  daysAbsentT2: number;
  daysAbsentT3: number;
  daysAbsentT4: number;
  daysAbsentYtd: number;
  lastActivityDate: string | null;
  lastActivity: string;
};
```

Why is `ItemRow` defined here (not in `crud.types.ts`)?

This is the **co-location principle**: things that change together should live together.

`ItemRow` and `mapItemDtoToRow` are tightly coupled:

- Every field in `ItemRow` must be populated by `mapItemDtoToRow`
- Every field in `mapItemDtoToRow`'s return must be in `ItemRow`
- If you add a column (new field in `ItemRow`), you must update `mapItemDtoToRow`
- If you rename a field in `ItemRow`, you must update `mapItemDtoToRow`

If `ItemRow` were in `crud.types.ts`, you'd have to open two files for every column change. Co-locating them means one file change, and TypeScript guides you to update both.

id: number — renamed from `StudentNumber`. TanStack Table uses `id` for row identity (the `getRowId` function). Using a generic name makes the table component reusable.

name: string — renamed from `StudentName`. camelCase convention for UI layer.

thresholdCrossed: string | null — `string | null` because the API can return `null` for this field. The `| null` forces every consumer to handle the null case. TypeScript will not let you call `.toUpperCase()` on `string | null` without a null check.

lastActivityDate: string | null — same pattern. A student with no recorded activity has `null` here.

The Mapper Function

```
export const mapItemDtoToRow = (dto: ItemDto): ItemRow => ({
  id: dto.StudentNumber,
  studentKey: dto.StudentKey,
  name: dto.StudentName,
  year: dto.CurrentYear,
  nsn: dto.NSN,
  homeClass: dto.HomeClass,
  coreClass: dto.CoreClass,
  house: dto.House,
  homeRoom: dto.HomeRoom,
  homeTeacher: dto.HomeTeacher,
  dean: dto.Dean,
  deputy: dto.Deputy,
  previousTermStatusId: dto.PreviousTermStatusId,
  currentTermStatusId: dto.CurrentTermStatusId,
  currentTermStatus: dto.CurrentTermStatusName,
  currentTermStatusColor: dto.CurrentTermStatusColor,
  previousTermStatus: dto.PreviousTermStatusName,
  previousTermStatusColor: dto.PreviousTermStatusColor,
  currentSummary: "",
  currentSummaryDetail: "",
  thresholdCrossed: dto.LastThresholdCrossedDate
    ? dayjs(dto.LastThresholdCrossedDate).format("DD/MM/YYYY")
    : null,
  daysAbsentT1: dto.DaysAbsentT1,
  daysAbsentT2: dto.DaysAbsentT2,
  daysAbsentT3: dto.DaysAbsentT3,
  daysAbsentT4: dto.DaysAbsentT4,
  daysAbsentYtd: dto.DaysAbsentYtd,
  lastActivityDate: dto.LastActivityDate
    ? dayjs(dto.LastActivityDate).format("DD/MM/YYYY")
    : null,
  lastActivity: dto.LastActivityName ?? "",
});
```

(dto: ItemDto): ItemRow => ({ ... }) — arrow function with an implicit return of an object literal. The parentheses around `{ ... }` are required — without them, JavaScript interprets `{` as the start of a function body, not an object literal.

```
// These are equivalent:
const fn = (dto) => ({ id: dto.StudentNumber }); // implicit return of object
const fn = (dto) => { return { id: dto.StudentNumber }; }; // explicit return
```

id: dto.StudentNumber — renames from domain-specific `StudentNumber` to generic `id`. This is the only place this renaming happens. If the backend renames the field, you update only this line.

currentTermStatus: dto.CurrentTermStatusName — renames from `CurrentTermStatusName` to `currentTermStatus`. The "Name" suffix is redundant in the UI layer.

currentSummary: "" — hardcoded empty string. This field is calculated elsewhere (not from the DTO). The `ItemRow` type includes it because the table column exists, but the data comes from a different source.

thresholdCrossed: dto.LastThresholdCrossedDate ? dayjs(...).format("DD/MM/YYYY") : null

This is a ternary expression:

```
condition ? value_if_true : value_if_false
```

If `dto.LastThresholdCrossedDate` is truthy (not null, not undefined, not empty string), format it with `dayjs`. Otherwise return null.

```
dayjs("2024-03-15T00:00:00").format("DD/MM/YYYY") → "15/03/2024"
```

lastActivity: dto.LastActivityName ?? ""

`??` is the nullish coalescing operator. If `dto.LastActivityName` is `null` or `undefined`, use `""` instead. This prevents `null` or `undefined` from appearing in the table cell.

The difference between `??` and `||`:

- `null ?? "" → ""` (correct)
- `"" ?? "default" → ""` (correct — empty string is a valid value)
- `"" || "default" → "default"` (wrong — treats empty string as falsy)

Use `??` when you only want to replace `null/undefined`, not falsy values like `0` or `""`.

How It's Used in the Hook

```
// useCrudPage.ts
const rows = useMemo<ItemRow[]>(
  () => items.map(mapItemDtoToRow),
```

```
[items],
);
```

`items.map(mapItemDtoToRow)` — calls `mapItemDtoToRow` for every item in the array. Returns a new array of `ItemRow` objects.

`useMemo` caches this result. Without `useMemo`, `.map()` would create a new array on every render — even renders caused by unrelated state changes (like the user typing in the search box). This would cause `TanStack Table` to re-initialize unnecessarily.

6.3 `buildEditContext.ts` — Line by Line

```
import type { ItemDto } from "../Services/crud.dtos";
import type { ItemEditContext } from "../Services/crud.types";

export const buildEditContext = (dto: ItemDto): ItemEditContext => ({
  mode: "update",
  studentActivityId: dto.LastActivityId,
  studentId: dto.StudentNumber,
  studentKey: dto.StudentKey,
  activityId: dto.LastActivityId,
  completedDate: dto.LastActivityDate ? new Date(dto.LastActivityDate) : new
Date(),
  completedBy: 0,
  notes: "",
  restrictedNotes: false,
});
```

What does this function do? When the user clicks "Edit" on a row, the app needs to pre-populate the modal form. This function builds the `ItemEditContext` from the `ItemDto` — the pre-populated data the modal needs.

mode: "update" — hardcoded because this function is only ever called when editing an existing item. The "create" context is built inline in the reducer with `mode: "create"` and empty defaults.

studentActivityId: dto.LastActivityId — the ID of the last recorded activity. Used in the update request: `PUT /api/NZAMP/UpdateStudentActivity` needs to know which activity to update.

activityId: dto.LastActivityId — pre-selects the same activity in the dropdown. The user can change it if needed.

completedDate: dto.LastActivityDate ? new Date(dto.LastActivityDate) : new Date()

- `dto.LastActivityDate` is an ISO string like `"2024-03-15T00:00:00"` or `null`
- `new Date("2024-03-15T00:00:00")` converts the ISO string to a JavaScript `Date` object
- `new Date()` (no argument) creates a `Date` for right now (today) — used as fallback when there's no existing date
- The form's date picker needs a `Date` object, not a string

completedBy: `0` — sentinel value meaning "not yet set". The `ItemForm` component will look up the logged-in user from `localStorage` and pre-populate the `TeacherSearch` component. `0` signals "use the default".

notes: `""` — empty string. The user types their own notes when editing. We don't pre-populate notes from the DTO because the DTO doesn't include the full notes text (it's not in the list response).

restrictedNotes: `false` — default to unrestricted. Same reason as notes.

6.4 Why Separate Functions Instead of Inline Logic?

You might wonder: why not just do this inline in the hook?

```
// Inline – works but not testable
const handleEdit = useCallback((itemId: number) => {
  const dto = items.find(i => i.StudentNumber === itemId);
  if (!dto) return;
  dispatch({
    type: "OPEN_EDIT",
    ctx: {
      mode: "update",
      studentActivityId: dto.LastActivityId,
      studentId: dto.StudentNumber,
      // ... 8 more fields
    }
  });
}, [items]);
```

Problems with inline logic:

1. The hook becomes longer and harder to read
2. You cannot test the context-building logic without testing the entire hook
3. If two places need to build an edit context, you duplicate the logic

With a separate function:

```
// In the hook – clean and readable
dispatch({ type: "OPEN_EDIT", ctx: buildEditContext(dto) });

// In a test – trivially testable
test("buildEditContext maps DTO correctly", () => {
  const dto = mockItemDto({ StudentNumber: 42, LastActivityDate: "2024-03-15" });
  const ctx = buildEditContext(dto);
  expect(ctx.studentId).toBe(42);
  expect(ctx.completedDate).toEqual(new Date("2024-03-15"));
  expect(ctx.mode).toBe("update");
});
```

6.5 How to Add a New Column

When you need to add a new column to the table:

1. **Add the field to `ItemRow`** in `mapItemDtoToRow.ts`:

```
export type ItemRow = {  
  // ... existing fields  
  newField: string; // add here  
};
```

2. **Map it in `mapItemDtoToRow`:**

```
export const mapItemDtoToRow = (dto: ItemDto): ItemRow => ({  
  // ... existing mappings  
  newField: dto.NewField ?? "", // add here  
});
```

TypeScript will show an error if you forget this step — `ItemRow` requires `newField` but the return object doesn't have it.

3. **Add a column definition** in `itemColumns.tsx`:

```
{  
  accessorKey: "newField",  
  header: ({ column }) => <TableHeaderButton label="New Field" column=  
    {column} />,  
  size: 8,  
},
```

4. **Optionally add to `FILTERABLE_COLUMNS`** if users should be able to filter by it:

```
{ id: "newField", label: "New Field", type: "string", valueType: "text" },
```

5. **Optionally add to `DEFAULT_COLUMN_VISIBILITY`** if it should be hidden by default:

```
newField: false,
```

That's it. TypeScript guides you through every step.

6.6 Unit Testing Pure Functions

Pure functions are the easiest code to test. Here's how you would test these:

```
// mapItemDtoToRow.test.ts
import { mapItemDtoToRow } from "../mapItemDtoToRow";

const mockDto = {
  StudentName: "John Smith",
  StudentNumber: 12345,
  CurrentYear: "10",
  NSN: "9876543210",
  HomeClass: "10A",
  // ... all required fields
  LastActivityDate: "2024-03-15T00:00:00",
  LastThresholdCrossedDate: null,
  LastActivityName: "Attendance Warning",
};

test("maps StudentNumber to id", () => {
  const row = mapItemDtoToRow(mockDto);
  expect(row.id).toBe(12345);
});

test("formats LastActivityDate to DD/MM/YYYY", () => {
  const row = mapItemDtoToRow(mockDto);
  expect(row.lastActivityDate).toBe("15/03/2024");
});

test("returns null for null LastThresholdCrossedDate", () => {
  const row = mapItemDtoToRow(mockDto);
  expect(row.thresholdCrossed).toBeNull();
});

test("renames StudentName to name", () => {
  const row = mapItemDtoToRow(mockDto);
  expect(row.name).toBe("John Smith");
});
```

No mocking, no setup, no teardown. Just call the function and check the output. This is the power of pure functions.

Next: [Chapter 7 — Columns](#)